

CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

1. Real-Time Face Detection and Recognition

Problem

A security system must process a live camera stream continuously. It should detect multiple faces, recognize registered persons using a trained dataset, display bounding boxes and identity labels, and operate with minimum processing delay.

Pseudocode

Algorithm: Real-Time Face Detection and Recognition using OpenCV

BEGIN

1. INITIALIZE SYSTEM

Import OpenCV library.

Import required libraries for numerical processing.

Load the trained face detection model.

Load the trained face recognition model.

Load the registered face dataset and corresponding person names.

Set recognition confidence threshold.

Set face detection parameters.

2. INITIALIZE CAMERA

Open the live camera using OpenCV.

Check whether the camera is opened successfully.

IF camera cannot be opened THEN

Display "Camera not available".

STOP the program.

END IF

3. START CONTINUOUS VIDEO PROCESSING

WHILE camera is running DO

3.1 CAPTURE FRAME

Read one frame from the camera.

IF frame is not successfully captured THEN

BREAK the loop.

END IF

3.2 PREPROCESS FRAME

Resize the frame to a suitable resolution

to reduce processing time.

Convert the frame from BGR to grayscale

if the selected face detector requires grayscale.

Apply noise reduction if necessary.

Improve image contrast or illumination
when required.

3.3 DETECT MULTIPLE FACES

Pass the preprocessed frame to the
face detection model.

Detect all faces present in the frame.

FOR each detected face DO

Obtain the bounding box:

x-coordinate

y-coordinate

width

height

Extract the detected face region
from the original frame.

3.4 PREPROCESS EACH FACE

Resize the extracted face to the
input size required by the recognition model.

Normalize the face image.

Align the face if facial landmarks
are available.

3.5 EXTRACT FEATURES

Pass the processed face to the
trained feature extraction model.

Generate the facial feature vector
or face embedding.

3.6 RECOGNIZE PERSON

Compare the extracted feature vector
with the feature vectors stored
in the registered dataset.

Calculate similarity or distance

between the live face and stored faces.

Find the best matching registered person.

IF matching score satisfies the
recognition threshold THEN

Assign the corresponding
registered person's name.

ELSE

Assign label "Unknown".

END IF

3.7 VISUALIZE RESULT

Draw a bounding rectangle around
the detected face.

Display the person's name or
"Unknown" above the bounding box.

Display recognition confidence
if required.

END FOR

3.8 DISPLAY FRAME

Display the processed frame
in a window.

3.9 CHECK EXIT CONDITION

Check whether the user has pressed
the designated exit key, such as 'q'.

IF exit key is pressed THEN

BREAK the loop.

END IF

3.10 REAL-TIME OPTIMIZATION

Process frames at a suitable resolution.

Avoid unnecessary image operations.

Detect faces at selected intervals if required.

Use lightweight models when running
on low-cost hardware.

Track already detected faces between
detection frames when appropriate.

END WHILE

4. RELEASE RESOURCES

Release the camera.

Close all OpenCV display windows.

Clear temporary resources.

5. TERMINATE PROGRAM

END

Flow of the Pseudocode

Start



Load Detection Model



Load Recognition Model



Load Registered Face Dataset



Open Live Camera



Capture Frame



Resize / Preprocess



Detect Multiple Faces



Extract Each Face



Normalize / Align Face



Extract Facial Features



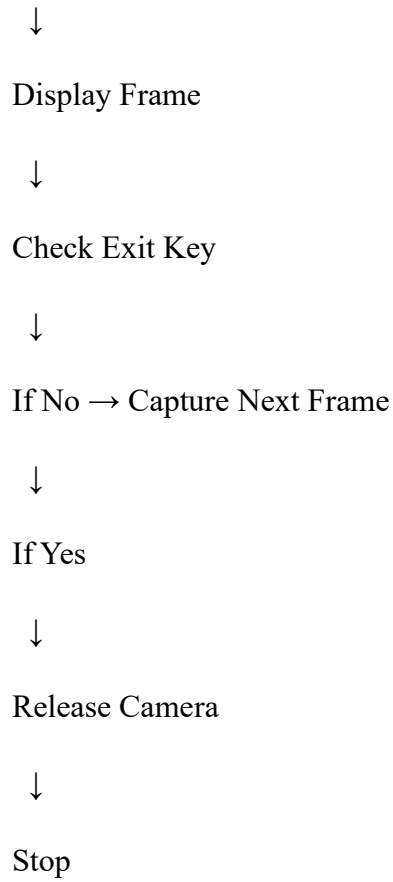
Compare with Registered Dataset



Recognized Person / Unknown



Draw Bounding Box + Identity



How the Constraints Are Satisfied

Real-timeoperation:

Frames are processed continuously using an efficient loop. Frame resizing and lightweight models reduce processing time.

Minimum processing delay:

Unnecessary operations are avoided, and the input resolution can be reduced. Face detection can also be performed periodically while tracking faces between detections.

Multiple faces:

The system processes every detected face individually within the same frame using a loop.

Recognition accuracy:

The extracted features are compared with the registered dataset using a recognition threshold. Low-confidence matches are classified as Unknown.

Visualization:

Each detected face receives a bounding box and identity label.

2. Vehicle Detection, Tracking and Counting

Problem

A traffic monitoring system must analyze a live video stream to detect vehicles entering a predefined ROI, track multiple vehicles, count vehicles crossing a virtual line, and display bounding boxes, tracking IDs, and vehicle count.

The system must avoid double counting, handle multiple vehicles, and maintain near real-time performance.

Pseudocode

Algorithm: Vehicle Detection, Tracking and Counting using OpenCV

BEGIN

1. INITIALIZE SYSTEM

Import OpenCV library.

Import required numerical and tracking libraries.

Load or initialize the vehicle detection method.

Initialize the multi-object tracker.

Define the Region of Interest (ROI).

Define the virtual counting line.

Initialize vehicle counter = 0.

Create an empty set called countedVehicles
to store IDs of vehicles that have already
crossed the counting line.

Set minimum vehicle area threshold.

2. INITIALIZE VIDEO SOURCE

Open the live traffic camera or video stream.

Check whether the video source is opened successfully.

IF video source cannot be opened THEN

 Display "Video source not available".

 STOP the program.

END IF

3. START VIDEO PROCESSING LOOP

WHILE video stream is available DO

3.1 CAPTURE FRAME

Read the next frame from the video.

IF frame is not successfully captured THEN

 BREAK the loop.

END IF

3.2 PREPROCESS FRAME

Resize the frame if necessary
to improve processing speed.

Apply Gaussian blur or another
noise reduction method.

Convert the frame to the required
color format.

3.3 SELECT REGION OF INTEREST

Extract only the predefined traffic ROI.

Ignore areas outside the ROI
because they are not required

for vehicle counting.

3.4 DETECT VEHICLES

Apply the selected vehicle detection
method to the ROI.

Detect cars, buses, trucks,
motorcycles, or other required vehicles.

FOR each detected vehicle DO

Obtain:

bounding box

center point

vehicle class

detection confidence

IF confidence is below the
selected threshold THEN

Ignore the detection.

END IF

IF vehicle area is below

minimum area THEN

Ignore the detection.

END IF

END FOR

3.5 UPDATE TRACKER

Send the valid vehicle detections
to the multi-object tracker.

The tracker assigns a unique tracking ID
to each vehicle.

Maintain the position of every vehicle
across consecutive frames.

Example:

Vehicle ID 1 → Frame 1 → Frame 2 → Frame 3

Vehicle ID 2 → Frame 1 → Frame 2 → Frame 3

Vehicle ID 3 → Frame 2 → Frame 3 → Frame 4

3.6 CALCULATE VEHICLE CENTERS

FOR each tracked vehicle DO

Calculate the center point
of its bounding box.

Store the previous center position.

Store the current center position.

3.7 CHECK VIRTUAL LINE CROSSING

Compare the previous center position
with the current center position.

IF vehicle moves from one side
of the virtual line to the other THEN

Check whether the vehicle ID
is already present in countedVehicles.

IF vehicle ID is NOT present THEN

Increase vehicle counter by 1.

Add vehicle ID to countedVehicles.

END IF

END IF

END FOR

3.8 DISPLAY VEHICLE INFORMATION

Draw the ROI boundary.

Draw the virtual counting line.

FOR each tracked vehicle DO

Draw bounding rectangle.

Display vehicle tracking ID.

Display vehicle class if required.

Display vehicle center point.

END FOR

Display total vehicle count.

3.9 DISPLAY FRAME

Show the processed frame
using an OpenCV window.

3.10 CHECK TERMINATION CONDITION

Check whether the user has pressed
the designated exit key, such as 'q'.

IF exit key is pressed THEN

BREAK the loop.

END IF

3.11 REAL-TIME OPTIMIZATION

Process only the required ROI.

Resize frames when appropriate.

Avoid unnecessary calculations.

Use an efficient vehicle detector.

Use tracking between detection operations
when appropriate.

END WHILE

4. RELEASE RESOURCES

Release the video source.

Destroy all OpenCV display windows.

Clear tracker resources.

5. DISPLAY FINAL COUNT

Display the final number of vehicles counted.

6. TERMINATE PROGRAM

END

Vehicle Counting Logic

The most important part of the system is avoiding double counting.

The logic can be represented as:

Vehicle detected



Assign Tracking ID



Track vehicle across frames



Calculate previous and current position



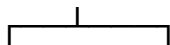
Did vehicle cross virtual line?



YES



Is ID already in countedVehicles?



YES NO



Ignore Count + 1



Store Vehicle ID

For example:

Frame 1 → Vehicle ID 5 → Above line

Frame 2 → Vehicle ID 5 → Near line

Frame 3 → Vehicle ID 5 → Below line



Line crossed



Count = Count + 1



Add ID 5 to countedVehicles

If Vehicle ID 5 appears again:

Vehicle ID 5



Already counted?



YES



Do not increase count

Therefore, **the same vehicle is counted only once.**

Flow of the Vehicle Detection System

Start



Initialize Vehicle Detector



Initialize Multi-Object Tracker



Define ROI



Define Virtual Counting Line



Set Vehicle Count = 0



Open Live Video



Capture Frame



Resize / Noise Reduction



Extract ROI



Detect Vehicles



Filter Invalid Detections



Update Tracker



Assign Tracking IDs



Calculate Vehicle Positions



Check Line Crossing



Check Already Counted ID



Update Vehicle Count



Draw Bounding Boxes + IDs



Display Count + ROI + Line



Check Exit Key



No → Process Next Frame



Yes



Release Video



Close Windows



Display Final Count



Stop

How the Constraints Are Satisfied

1. Avoid Double Counting

Every vehicle receives a unique tracking ID. Once that ID crosses the virtual line and is counted, it is stored in `countedVehicles`. The same ID cannot increase the count again.

2. Handle Multiple Vehicles

All valid detections in every frame are processed using a loop. The tracker maintains a separate ID and position for each vehicle.

3. Near Real-Time Performance

The system can improve speed by:

- Processing only the ROI.
- Resizing frames.
- Using lightweight detection models.
- Filtering small detections early.
- Tracking vehicles between detection operations.
- Avoiding unnecessary image processing.

4. Accurate Line Crossing

The previous and current centroid positions are compared. A vehicle is counted only when its movement changes from one side of the virtual line to the other.

5. Visualization

The output frame contains:

ROI

Virtual Counting Line

Vehicle Bounding Box

Vehicle Tracking ID

Vehicle Count